

课程笔记

Gemini Agentic Vision: 模型基础能力与 Post-Training

基于公开课程资料整理

2025

Gemini 3 Flash Agentic Vision

视频作者/频道: 五道口纳什

发布日期: 2025

视频时长: 约 16 分钟

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言：Agentic Vision 的启发意义	2
2 Agentic Vision 的核心机制	2
2.1 从”静态观察”到”主动调查”	2
2.2 唯一的 Tool: Code Execution	2
2.3 ReAct 循环: Think → Act → Observe	3
3 与 SAM3 Agent 的对比	3
3.1 显式构建 vs. 自主涌现	3
3.2 Visual Prompting 的演进	3
3.3 本章小结	3
4 基础能力与 Post-Training 的关系	4
4.1 两阶段范式	4
4.2 Trigger Prompt 的角色	4
4.3 本章小结	4
5 典型应用场景分析	4
5.1 场景一：精确计数（手指/踏板）	4
5.2 场景二：高密度表格解析 + 计算	5
5.3 场景三：物品分类与标注	5
5.4 场景四：动物生命周期图表	5
5.5 本章小结	5
6 抓包分析：API 层面的理解	6
7 对 Agent Research 的启示	6
8 总结与延伸	6
8.1 拓展阅读	7

1 引言：Agentic Vision 的启发意义

本期介绍 Google 最新发布的 Gemini 2.5 Flash Agentic Vision 功能。这项工作将视觉推理与代码执行相结合，让模型能够主动“调查”图像——而非被动地“看一眼就猜”。

Modern Agents 与 Agentic RL Training 的关系

讲者将两个系列做了明确定位：

- **Modern Agents**：侧重推理、多轮工具调用、Novel Prompting 技巧
- **Agentic RL Training**：侧重算法、系统架构、Post-Training

Agentic Vision 恰好连接了两者——它展示的多轮推理效果，启发了如何通过 Post-Training 来强化和激励模型的基础能力。

2 Agentic Vision 的核心机制

2.1 从“静态观察”到“主动调查”

传统 VLM 以单一静态视角处理视觉输入——如果遗漏了细微细节，只能被迫猜测。Agentic Vision 将图像理解从静态行为转变为智能体过程：

Agentic Vision 的三大创新

- **视觉推理 + 代码执行**：模型通过写 Python 代码来操作图像
- **多轮 Tool Call**：Think → Act → Observe 的 ReAct 循环
- **视觉草图 (Visual Scratchpad)**：模型自主生成标注图来辅助推理

2.2 唯一的 Tool：Code Execution

Agentic Vision 的 Tool 设计极其简洁——只有一个**代码执行工具** (Code Execution on Images)。通过写代码，模型可以实现：

- **Bounding Box**：画检测框
- **Crop**：裁切图像区域
- **Rotate/Zoom**：旋转和放大
- **Annotate**：在图上打标注（文字、箭头、编号）
- **Draw**：绘制辅助线条
- **Save**：保存处理后的图像

关键洞察：BBox 不是外部 CV 模型产生的

Agentic Vision 中的 Bounding Box 坐标是 **Gemini 2.5 Flash** 自己产生的——它在写代码时直接把 BBox 坐标传进去。这是模型**原生的基础视觉能力**（检测、分割、识别），而非依赖外部的目标检测模型。

2.3 ReAct 循环：Think → Act → Observe

Agentic Vision 的工作流程是一个标准的 ReAct (Reasoning + Acting) 过程：

1. **Think**：分析用户问题，规划视觉操作步骤
2. **Act**：生成 Python 代码，调用 Code Execution Tool
3. **Observe**：查看代码执行结果（处理后的图像），决定是否需要进一步操作
4. 循环直到得出最终答案

这是 Post-Training 激励出来的

ReAct 循环不是通过外部框架硬编码的，而是通过 Post-Training（强化学习）激励模型自发形成的多轮工具调用行为。模型学会了在什么时候该 zoom in、该 crop、该 annotate。

3 与 SAM3 Agent 的对比

3.1 显式构建 vs. 自主涌现

上一期介绍的 SAM3 Agent 也实现了 Visual Prompting，但方式截然不同：

维度	SAM3 + VLM	Gemini Agentic Vision
视觉工具	SAM3（专用分割模型）	Code Execution（通用）
架构	显式构建的 Agent Pipeline	Post-Training 激励出的行为
视觉草图	由 SAM3 生成 Mask 叠加图	模型自主写代码生成
BBox 来源	外部模型/工具	模型原生能力
灵活性	受限于定义的 Tool	理论上可完成任意图像操作

表 1: SAM3 Agent 与 Gemini Agentic Vision 对比

3.2 Visual Prompting 的演进

两种方案都产生了”视觉草图”（Visual Scratchpad），但来源不同：

- SAM3 Agent：人类**显式设计**了分割 → 标注 → 返回 VLM 的流程
- Agentic Vision：模型**自主决定**需要什么样的视觉草图来辅助推理

3.3 本章小结

从显式构建到自主涌现，Agentic Vision 代表了 Visual Prompting 技术的进化方向——让模型自己决定如何”看”图，而非人类预设好观察方式。

4 基础能力与 Post-Training 的关系

4.1 两阶段范式

Agentic Vision 揭示了一个重要的训练范式：

基础能力 + Post-Training 释放

1. **预训练阶段**：训练模型的基础视觉能力——检测、分割、识别。这些能力是”沉睡”的，模型已经知道如何做，但不知道何时做、如何组合。
2. **Post-Training 阶段**：封装 Tool (如 Code Execution)，通过强化学习激励模型形成**多轮工具调用**的行为模式，将基础能力”释放”出来。

这一范式对 AI Agent Research 有两层启发：

1. **对 Prompting/Agent 设计者**：通过巧妙的 Visual Prompting 和 Tool 设计，可以在不改变模型参数的情况下，让之前不可行的任务变得可行
2. **对模型训练者**：基础能力的质量决定了 Post-Training 后 Agent 的上限

4.2 Trigger Prompt 的角色

在 Gemini 的官方 Demo 中，Prompt 中包含一些”触发词”来激发 Agentic Vision 能力：

- ”crop out all ...”
- ”zoom in to see ...”
- ”annotate on the image ...”

Trigger Prompt 是过渡性设计

如果模型训练足够好，理论上不需要这些 Trigger Prompt——模型应该能够自主判断何时需要 zoom in、何时需要 annotate。Trigger Prompt 的存在说明当前模型的 Post-Training 还有优化空间。

4.3 本章小结

Agentic Vision 的核心启示是：Agent 的能力上限由模型的基础能力决定，Post-Training 的作用是将这些基础能力组合、释放为完整的多轮推理行为。

5 典型应用场景分析

5.1 场景一：精确计数（手指/踏板）

任务：数一幅图中有多少根手指/踏板。

Agentic Vision 的做法：

1. 模型识别出所有目标，生成每个目标的 BBox 坐标（原生能力）

2. 写代码在图上画出编号标注框，形成视觉草图
3. 基于标注后的图，确认数量

关于“Tricky”的讨论

讲者指出这个示例有些“tricky”——模型在输出 BBox 时其实已经知道了目标数量（BBox 数 = 手指数）。标注和二次确认更多是一种“视觉验证”而非真正的推理突破。

5.2 场景二：高密度表格解析 + 计算

任务：解析图中的表格数据，进行归一化计算并可视化。

传统模型的问题：VLM 在多步视觉算术中经常出现幻觉（hallucination），尤其是浮点数运算。

Agentic Vision 的解决方案：

1. 使用原生图表理解能力提取表格数据
2. 将计算“下载”到确定性的 Python 环境——在代码中完成归一化等数学运算
3. 在 Python 中完成 Matplotlib 可视化

计算卸载（Computation Offloading）

Agentic Vision 的一个重要设计模式是**将精确计算从模型推理中卸载到代码执行环境**。模型负责理解和规划，Python 负责精确计算。这有效规避了 LLM 固有的数值计算不稳定性。

5.3 场景三：物品分类与标注

任务：将图中的不同物品分类，用不同颜色的箭头标注到对应的容器中。

这是一个综合性任务，需要模型同时具备：

- 物体识别（知道这是什么）
- 分类推理（应该放到哪个类别）
- 视觉标注（画箭头连接物品和容器）

讲者测试发现，模型在分类逻辑上仍有瑕疵（如将某些物品错误分类），但整体的视觉交互能力令人印象深刻。

5.4 场景四：动物生命周期图表

任务：从图中裁切出所有动物，以它们的图片作为 icon，在 Matplotlib 图表中按生命周期长度排序展示。

这个案例涉及**非常多轮的工具调用**——裁切、排序、绘图等步骤需要多次 Think-Act-Observe 循环。最终生成的图表兼具信息性和美观性。

5.5 本章小结

Agentic Vision 的应用场景涵盖了精确计数、表格解析、物品分类、图表生成等多种任务。其核心模式是“模型负责理解与规划 + 代码负责精确执行”。

6 抓包分析：API 层面的理解

讲者通过 HTTP 请求/响应的 Hook 对 Gemini API 进行了抓包分析：

- **Request Body**：包含 Tools 定义（Code Execution）和用户输入（图片 + 文本）
- **Response Body**：包含多个 Parts，每个 Part 可能是：
 - `executable_code`：模型生成的 Python 代码
 - `code_execution_result`：代码执行的结果（包括生成的图片）
 - `text`：模型的文本推理输出
- 前端展示的每一步过程，本质上就是在可视化这些 Parts

Gemini App vs AI Studio

截至视频录制时，Gemini App（消费者版本）尚未提供 Code Execution Tool。要体验 Agentic Vision，需要使用 AI Studio（开发者工具）。

7 对 Agent Research 的启示

讲者分享了对 AI Agent 研究方向的个人思考：

1. **纯 Scaffold Agent 的意义在下降**：做一个通用的 Agent 表现（solid performance）来发论文，意义越来越小
2. **聚焦特定 Task/Tool**：更有价值的研究方向是——针对某个具体任务或工具，通过 Agent/Prompting 设计让之前不可行的变得可行、之前有效果的大幅提升
3. **Visual Prompting 是一个好方向**：它将视觉信息编码为 Agent 可利用的形式，是连接视觉模型和语言模型的桥梁
4. **Agentic RL Training**：在预训练中建立基础能力，在 Post-Training 中通过强化学习激励多轮工具调用，释放基础能力

8 总结与延伸

Gemini Agentic Vision 是一项将模型基础视觉能力与 Agentic 多轮推理结合的创新工作。它的核心贡献不仅在于功能本身，更在于揭示了一条清晰的技术路线：

1. **模型层**：预训练获得基础视觉能力（检测、分割、识别）+ Open World Knowledge
2. **工具层**：封装通用的 Code Execution Tool
3. **行为层**：通过 Post-Training 激励 ReAct 式的多轮工具调用
4. **应用层**：模型自主生成 Visual Scratchpad，基于视觉证据做推理

8.1 拓展阅读

- Gemini Agentic Vision 官方 Blog 及 12 个示例 Demo
- AI Studio: 可在线体验 Code Execution Tool
- ReAct 论文 (Yao et al., 2023): Reasoning + Acting 的理论基础